

** 농구선수 포지션 데이터 활용한 KNN 모델 실습 예제 **

```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

```
df = pd.read_csv('basketball_stat.csv')
print(df.head())
print(df.info())
print(df['Pos'].value_counts())
```

```
sns.lmplot(x='TRB',y='3P',data=df, fit_reg=False,
           scatter_kws={'s':150,markers=['o','x'], hue='Pos'})
plt.title('TRB and 3P in 2d plane')
plt.show()
```

total
Rebound

('TRB', '3P' 특성 항목을 이
용한 'Pos' 분류 시각화)

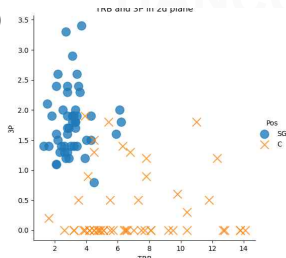
Total Rebound percentage

Block
Percentage

	Player	Pos	3P	2P	TRB	AST	STL	BLK
0	Alex Abrines	SG	1.4	0.6	1.3	0.6	0.5	0.1
1	Steven Adams	C	0.0	4.7	7.7	1.1	1.1	1.0
2	Alexis Ajinca	C	0.0	2.3	4.5	0.3	0.5	0.6
3	Chris Andersen	C	0.0	0.8	2.6	0.4	0.4	0.6
4	Will Barton	SG	1.5	3.5	4.3	3.4	0.8	0.5

```
C    50
SG   50
Name: Pos, dtype: int64
```

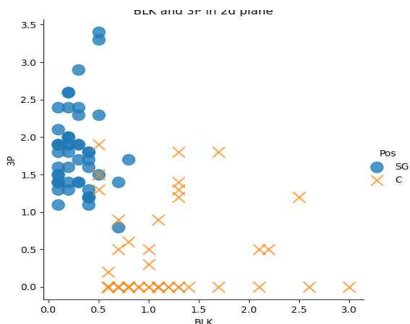
C : 센터
SG : 슈팅가드



1. 농구선수 포지션 데이터 특성 시각화 및 추출

```
sns.lmplot(x='BLK',y='3P',data=df, fit_reg=False,
            scatter_kws={'s':150},markers=['o','x'], hue='Pos')
plt.title('BLK and 3P in 2d plane')
plt.show()
```

('BLK', '3P' 특성 항목을 이용
한 'Pos' 분류 시각화)



#시각화로 '3P', 'TRB', 'BLK' 컬럼 데이터가 Pos 구분에
적절한 데이터로 판단

#데이터 가공 : 불필요한 '2P','AST','STL' 컬럼 데이터 삭제
df.drop(['2P','AST','STL'],axis=1, inplace=True)
print(df.head())

2. 특성 정리된 데이터 활용한 KNN 모델 k-fold 교차 검증 (최적 하이퍼파라미터 찾기 ==> 이웃(k) 개 설정 값)

~~~~~ 앞 PPT 에서 정리된 df 데이터프레임 활용 ~~~~~

# 훈련 / 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split
```

```
train, test = train_test_split(df, test_size=0.2, random_state=45) # 20 비율로 훈련셋, 테스트셋 분리
```

```
print(train.shape[0]) # 80
```

```
print(test.shape[0]) # 20
```

# k-fold 교차 검증으로 최적의 Knn 파라미터 ( k 값 ) 찾기

# cross\_val\_score() 활용

# 최적의 k를 찾기위한 교차 검증 수행할 k의 범위를

# 3 부터 학습 데이터 절반까지 설정, 홀수로 설정이 적절

```
from sklearn.neighbors import KNeighborsClassifier
```

```
from sklearn.model_selection import cross_val_score
```

```
max_k_range = train.shape[0] // 2
```

```
k_list = []
```

```
for i in range(3, max_k_range, 2):
```

```
    k_list.append(i)
```

```
print(k_list) # k 값 리스트
```

[3, 5, 7, 9, 11, 13, 15, 17, 19, 21, 23, 25, 27,  
29, 31, 33, 35, 37, 39]

# 데이터 추출

```
x_train = train[['3P','BLK','TRB']]
```

# 훈련데이터 특성

```
y_train = train[['Pos']]
```

# 훈련데이터 타겟

## 2-1) k 값 바뀌가면서 knn 모델 k-fold 교차 검증

```
cross_validation_scores = []
for k in k_list:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, x_train, y_train.values.ravel(), cv=10, scoring='accuracy')
    cross_validation_scores.append(scores.mean())
```

```
print(cross_validation_scores)
```

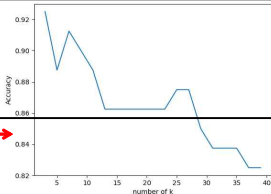
훈련데이터를 10번 교차 검증한:  
정확도 반환

각 k 값에 대한 10번  
의 교차검증에 대한  
평균값을 list에 저장

k-fold 교차검증  
에 대한 별참자료  
참조

```
[0.925, 0.8875, 0.9125, 0.9, 0.8875, 0.8625, 0.8625, 0.8625, 0.8625, 0.8625, 0.875, 0.875, 0.85, 0.8375, 0.8375, 0.8375, 0.825, 0.825]
```

```
plt.plot(k_list, cross_validation_scores)
plt.xlabel('number of k')
plt.ylabel('Accuracy')
plt.show()
```



( k=3 일때 가장 높은  
정확도 )

### 3. test 데이터 활용한 Knn 모델 평가 및 예측

~~~~~ 앞 PPT 에서 정리된 df 데이터프레임 활용 ~~~~~

훈련 / 테스트 데이터 분리

```
from sklearn.model_selection import train_test_split
```

```
train, test = train_test_split(df, test_size=0.2, random_state=49) # 20 비율로 훈련셋, 테스트셋 분리
```

```
print(train.shape[0]) # 80
```

```
print(test.shape[0]) # 20
```

```
from sklearn.neighbors import KNeighborsClassifier
```

```
knn = KNeighborsClassifier(n_neighbors=3)
```

데이터 추출

```
x_train = train[['3P','BLK','TRB']]
```

```
y_train = train[['Pos']]
```

KNN 모델 학습

```
knn.fit(x_train, y_train.values.ravel())
```

교차 검증으로 찾은 k 값 3으로 설정

테스트 데이터에서 분류를 위해 사용될 속성을 지정

```
x_test = test[['3P','BLK','TRB']] # x_test 데이터로 예측
```

```
y_test = test[['Pos']] # 분류 실제 값
```

타깃 데이터 1차원 배열로
변환하여 전달

3-1) test 데이터 활용한 Knn 모델 평가 및 예측

테스트 데이터 예측 시작

```
pred = knn.predict(x_test)
```

```
print('예측 결과 : ', pred)
```

```
['SG' 'C' 'C' 'C' 'SG' 'C' 'SG' 'SG' 'SG' 'SG' 'SG' 'C' 'SG' 'C' 'SG' 'C'
 'C' 'C' 'C' 'C']
```

모델 예측 정확도 출력

```
print(y_test.values) # numpy (20,1) 2차원배열
```

```
print(y_test.values.ravel()) # numpy 1차원 배열 데이터
```

```
from sklearn.metrics import accuracy_score
```

accuracy_score() : confusion matrix(혼동행렬) 활용 정확도 계산

```
print('accuracy : ' + str(accuracy_score(y_test.values.ravel(), pred)) ) # 0.95
```

```
print('acc : ', knn.score(x_test, y_test)) # 0.95
```

실제 데이터와 예측 데이터 Dataframe 변환 출력

```
comparison = pd.DataFrame({'prediction':pred, 'truth value':y_test.values.ravel()})
```

```
print(comparison.head())
```

| | prediction | truth value |
|---|------------|-------------|
| 0 | SG | SG |
| 1 | C | C |
| 2 | C | C |
| 3 | C | C |
| 4 | SG | SG |

Thank You



HANCOM

Template Visual Guide, version 1.0

© Hancorn Inc. / Pangyo, February 2019